

SESSION 5 — Histogram (Distribution Plot)

What is a Histogram?

A histogram is used to visualize the **distribution of numerical data**.

Why Histogram is Used

It helps to understand:

- Data spread
 - Frequency distribution
 - Skewness (left/right)
 - Presence of outliers
-

Example Use Cases

- Customer age distribution
 - Salary distribution
 - Exam scores
-

Key Concept

Histogram groups data into intervals called **bins**

✓ plt.hist()

Syntax

```
plt.hist(x, bins=10, density=False, color=None, edgecolor=None)
```

Parameters (Important)

1. x

Data values (numerical list/array)

2. bins (default = 10)

Number of intervals

Higher bins → more detail

Lower bins → more general view

3. density (default = False)

False → shows frequency (count)

True → shows probability density

4. color (default = None)

Defines color of bars

5. edgecolor (default = None)

Color of bar borders

Use Case

- Analyze distribution shape
- Identify concentration of values

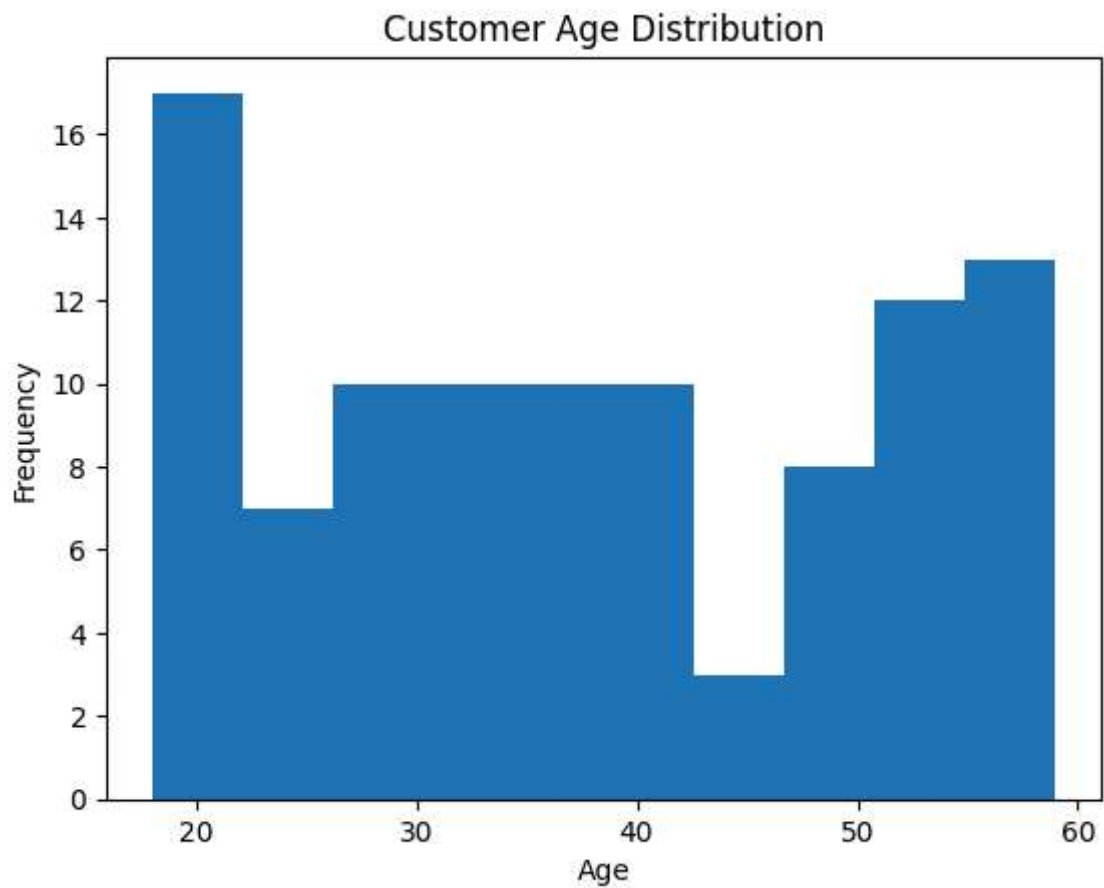
```
import matplotlib.pyplot as plt
import numpy as np

# Generate sample data (customer ages)
np.random.seed(0)
ages = np.random.randint(18, 60, 100)

plt.hist(ages)

plt.title("Customer Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")

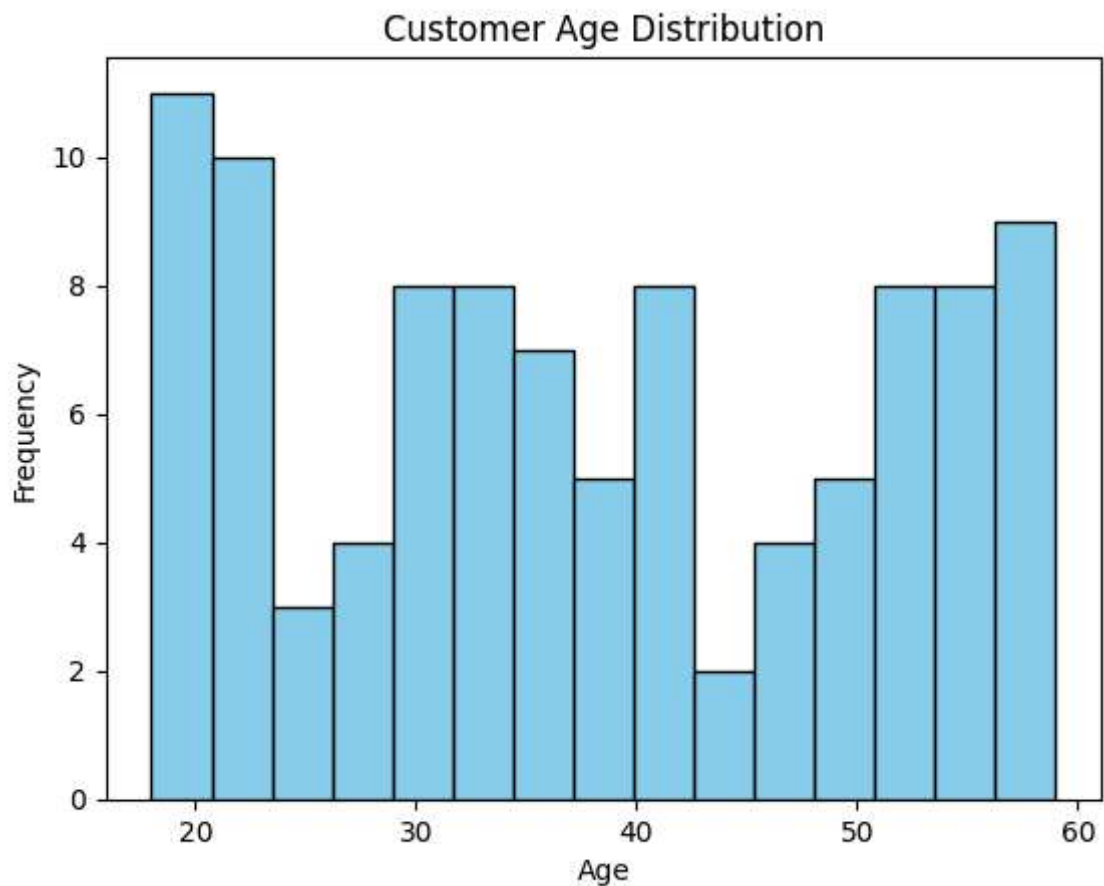
plt.show()
```



✓ Custom Histogram

Control bins and styling for better clarity

```
plt.hist(  
    ages,  
    bins=15,  
    color='skyblue',  
    edgecolor='black'  
)  
  
plt.title("Customer Age Distribution")  
plt.xlabel("Age")  
plt.ylabel("Frequency")  
  
plt.show()
```



✓ Density vs Frequency

density = False (default)

Shows:

Number of observations in each bin

density = True

Shows:

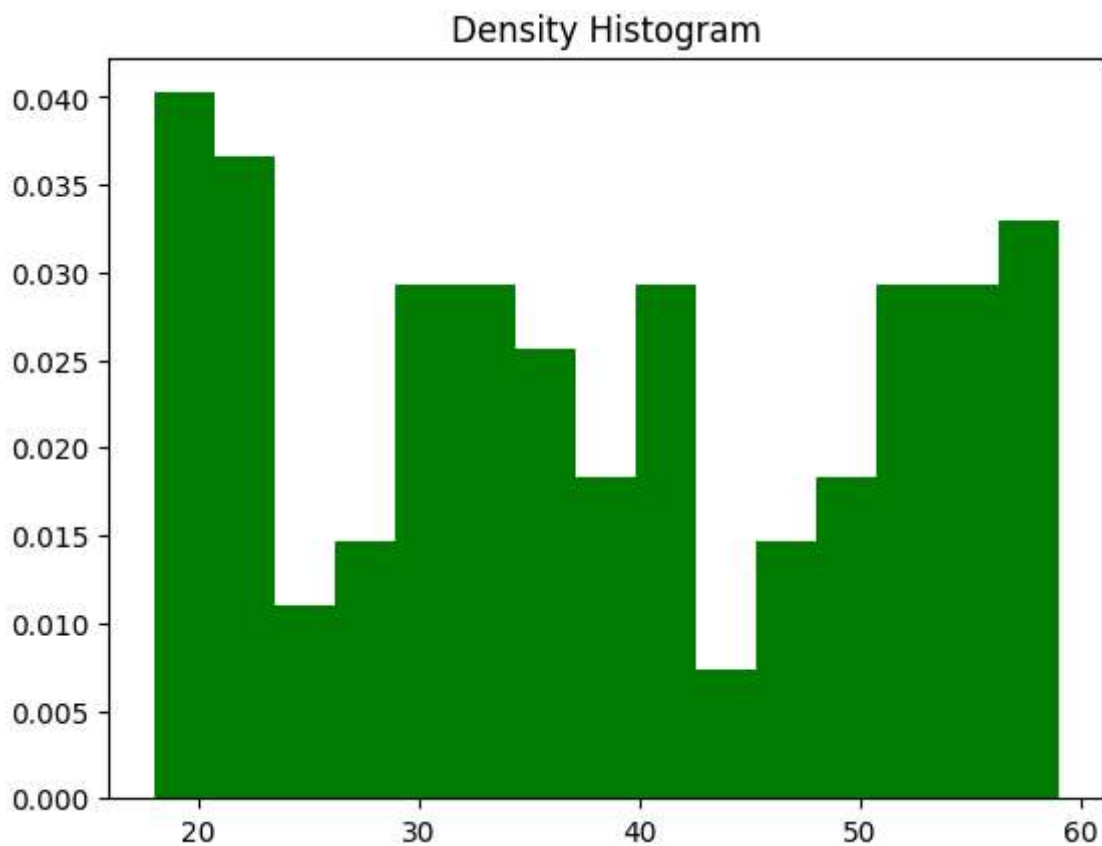
Probability distribution

When to Use

density=True:

- Comparing distributions
- Statistical analysis

```
plt.hist(ages, bins=15, density=True, color='green')  
  
plt.title("Density Histogram")  
plt.show()
```



SESSION 6 — Scatter Plot

What is a Scatter Plot?

A scatter plot shows **relationship between two numerical variables**.

Why Use Scatter Plot

- Identify correlation
- Detect patterns
- Find trends

Example Use Cases

- Advertising spend vs sales
- Study hours vs marks

- Price vs demand

✓ plt.scatter()

Syntax

`plt.scatter(x, y, color=None, s=None, label=None)`

Parameters

1. x
X-axis values
 2. y
Y-axis values
 3. color (default = None)
Point color
 4. s (default = None)
Size of points
 5. label (default = None)
Used in legend
-

Use Case

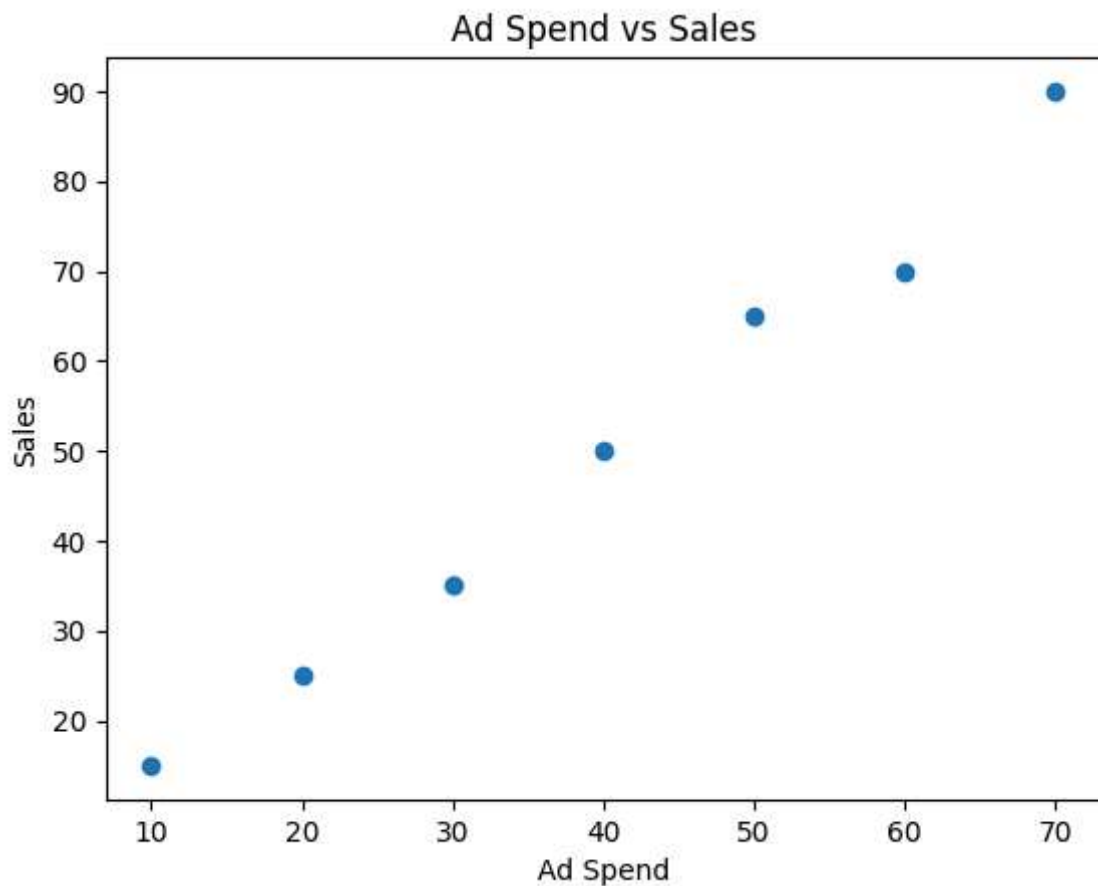
- Relationship analysis
- Pattern detection

```
# Sample data
ad_spend = [10,20,30,40,50,60,70]
sales = [15,25,35,50,65,70,90]

plt.scatter(ad_spend, sales)

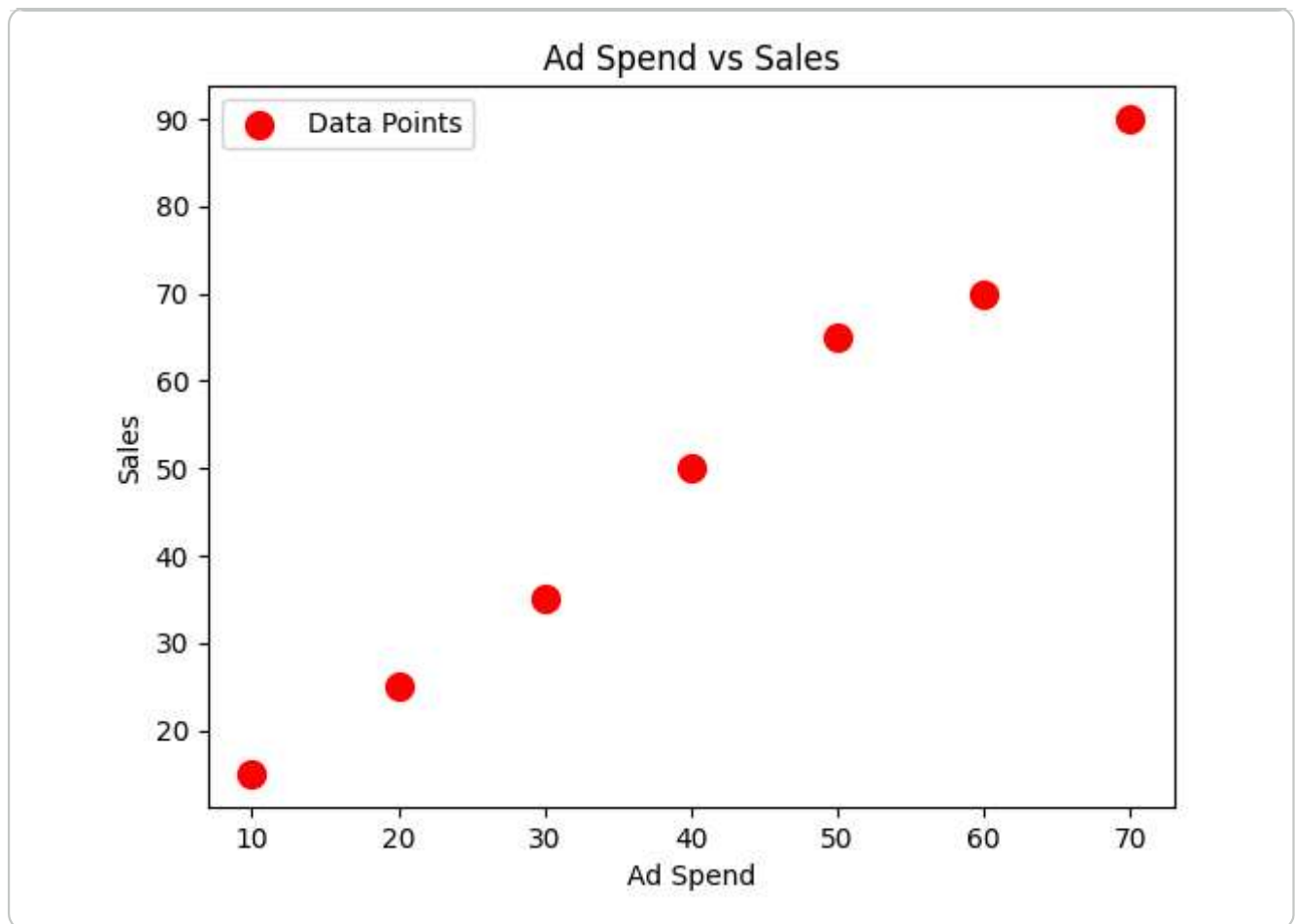
plt.title("Ad Spend vs Sales")
plt.xlabel("Ad Spend")
plt.ylabel("Sales")

plt.show()
```



✓ Customized Scatter Plot

```
plt.scatter(  
    ad_spend,  
    sales,  
    color='red',  
    s=100,  
    label="Data Points"  
)  
  
plt.title("Ad Spend vs Sales")  
plt.xlabel("Ad Spend")  
plt.ylabel("Sales")  
  
plt.legend()  
  
plt.show()
```



Trend Line (Regression Line)

Why Add Trend Line?

To understand:

- Direction of relationship
- Strength of relationship

Method

Use NumPy:

`np.polyfit()`

✓ `np.polyfit()`

Syntax


```
np.polyfit(x, y, degree)
```

Parameters

$x \rightarrow$ independent variable

$y \rightarrow$ dependent variable

degree:

1 $\rightarrow$ linear fit (straight line)

Output

Returns slope and intercept

```
import numpy as np

# Fit line
m, b = np.polyfit(ad_spend, sales, 1)

# Create line
line = [m*x + b for x in ad_spend]

# Plot scatter

plt.scatter(ad_spend, sales, color='blue')

# Plot trend line
plt.plot(ad_spend, line, color='black')

plt.title("Ad Spend vs Sales with Trend Line")
plt.xlabel("Ad Spend")
plt.ylabel("Sales")

plt.show()
```

